ICS 344 - 02

MOAYED ALJADDAWI - 202248800

DVSA URL: http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/

AWS Region: us-east-1

02/05/2026

Lesson #8: Logic Vulnerability

Lesson #8: Logic Vulnerability

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a logic vulnerability, specifically a race condition, in the DVSA order-processing workflow. The affected components are the /dvsa/order API endpoint, the DVSA-ORDER-BILLING Lambda function, the DVSA-ORDER-UPDATE Lambda function, and the DVSA-ORDERS-DB DynamoDB table.

The security impact is financial loss and inconsistent order state. An attacker can submit a billing request and an update request close together, causing the system to charge the user for a smaller quantity while the final stored order contains a larger quantity. In my test, the order started with quantity 1 and total amount around $33, but after the race condition the DynamoDB record showed item quantity 10 while totalAmount remained 33.

Part 2) Why This Works / Root Cause

The vulnerability works because the backend does not enforce strict workflow sequencing between order updates and billing. The system allows an order to be updated while billing is being processed. This creates a race window where the billing function calculates the total based on the original quantity, but the update function changes the item quantity before the final order state is stored.

The root cause is missing synchronization and weak state validation. The billing function did not lock the order immediately when payment started, and the update function allowed changes unless the order was already considered paid. This allowed two concurrent requests to create an inconsistent final state.

Part 3) Environment and Setup

The test was performed in a controlled DVSA deployment in AWS **us-east-1.**

Environment details used in the demonstration:

- **DVSA API endpoint used:**
  - https://n1pq1eb7kd.execute-api.us-east-1.amazonaws.com/dvsa/order
- **Main AWS services involved:**
  - Amazon API Gateway
  - AWS Lambda
  - Amazon DynamoDB
- **Lambda functions involved:**
  - DVSA-ORDER-BILLING
  - DVSA-ORDER-UPDATE
- **DynamoDB table involved:**
  - DVSA-ORDERS-DB
- **Tools used:**
  - DVSA website
  - Browser DevTools
  - PowerShell
  - AWS Lambda Console

  - AWS DynamoDB Console
- **Test context:**
  - A normal authenticated DVSA user account was used.
  - A valid token was used in PowerShell requests.
  - The test began with a normal cart containing one item.

Part 4) Reproduction Steps

1. Open the DVSA website and log in using a normal user account.
2. Add one product to the cart with quantity 1.
3. Confirm the cart total. In my test, the cart showed quantity 1 and total price around $33.
4. Open Browser DevTools.
5. Go to Network → Fetch/XHR.
6. Click checkout to create a new order.
7. Capture the action: "new" request and response.
8. Confirm that the backend created a new order. The response returned:
   ```
   {
    "status": "ok",
    "msg": "order created",
    "order-id": "a7f1eddf-367f-474f-9d40-a9a303744bc5"
   }
   ```
9. In PowerShell, prepare two requests using the same order ID.
10. Prepare the billing request:
    ```
    $billingBody = @{
     action = "billing"
     "order-id" = $orderId
     data = @{
      ccn = "4242424242424242"
      exp = "02/29"
      cvv = "123"
     }
    } | ConvertTo-Json -Compress
    ```
11. Prepare the update request. This request uses the same order ID but changes item 1012 to **quantity 10:**
    ```
    $updateBody = @{
     action = "update"
     "order-id" = $orderId
     items = @{
      "1012" = 10
     }
    } | ConvertTo-Json -Compress
    ```
12. Send the billing request and the update request close together using PowerShell background jobs:
    ```
    $billingJob = Start-Job -ScriptBlock {
     param($api, $headers, $body)
     Invoke-RestMethod -Uri $api -Method Post -Headers $headers -Body $body
    } -ArgumentList $API, $headers, $billingBody
    ```

```
Start-Sleep -Milliseconds 50

$updateJob = Start-Job -ScriptBlock {
  param($api, $headers, $body)
  Invoke-RestMethod -Uri $api -Method Post -Headers $headers -Body $body
} -ArgumentList $API, $headers, $updateBody

Receive-Job $billingJob -Wait
Receive-Job $updateJob -Wait
```

13. Observe that both requests succeeded before the fix:
    - Billing returned status: ok and charged amount 33.
    - Update returned status: ok and msg: cart updated.
14. Open AWS DynamoDB.
15. Go to:
    DynamoDB → Explore items → DVSA-ORDERS-DB
16. Search for the exploited order ID.
17. Open the order item and inspect:
    - itemList
    - totalAmount
    - orderStatus
18. Confirm the inconsistent state:
    - itemList → 1012 = 10
    - totalAmount = 33
    - orderStatus = 200

This proves the user was charged for one item while the final order stored ten items.

Part 5) Evidence and Proof

The vulnerability was proven using the DVSA cart, the order creation response, PowerShell race-condition requests, and the DynamoDB final order record.

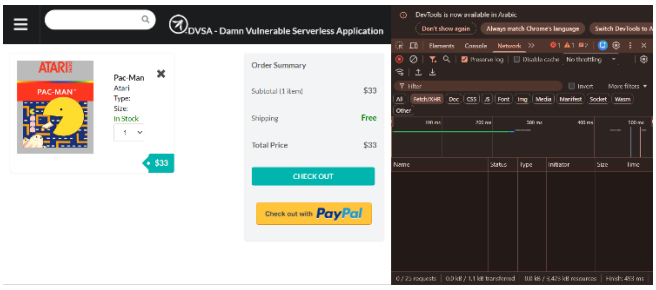

Figure 1. Initial cart before the race-condition test showing quantity 1 and total price $33.

This proves the order started with one item.

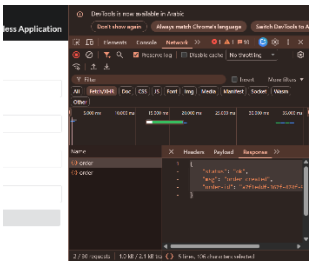

Figure 2. New order created for the race-condition test.

This proves a fresh order was created before running the exploit.

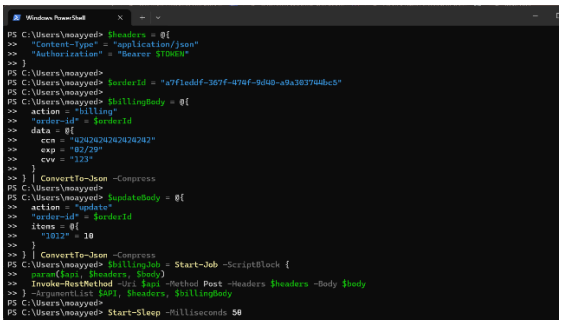

Figure 3. PowerShell setup showing billing and update requests using the same order ID.

This proves both requests targeted the same order, and the update request attempted to change the item quantity to 10.

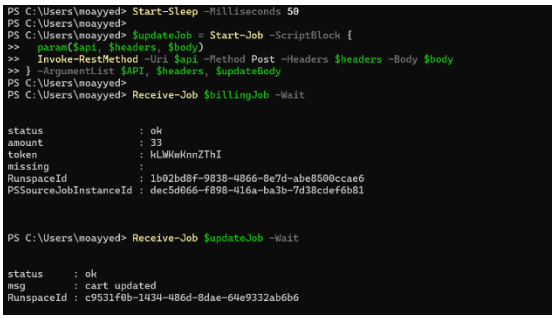

Figure 4. Concurrent billing and update requests both succeeded before remediation.

This proves both the billing and update requests are successful.

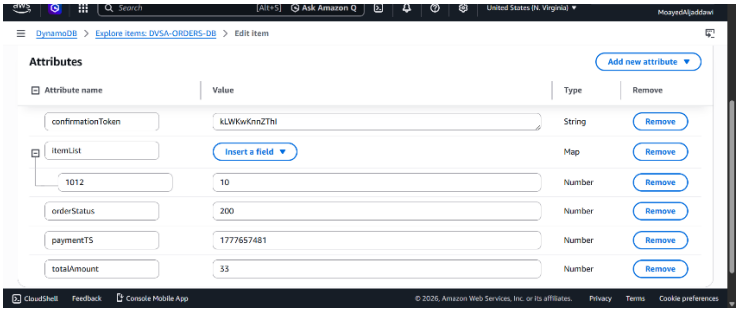

Figure 5. DynamoDB record after exploitation showing quantity 10 while total amount remained 33.

This is the strongest proof. The final stored order contained ten items, but the charged amount remained the price for one item. This confirms the logic race condition.

Part 6) Fix Strategy / Probable Mitigation

The fix should enforce strict order workflow control. Once billing starts, the order should no longer be editable. This can be done by locking the order immediately at the beginning of billing and rejecting update requests unless the order is still open.

The mitigation strategy was:

1. In order_billing.py, allow billing only when orderStatus == 100.
2. Immediately change orderStatus from 100 to 200 when billing starts.
3. Use a DynamoDB ConditionExpression to ensure the order is only locked if it is still open.
4. In update_order.py, allow updates only when orderStatus == 100.
5. Add a DynamoDB ConditionExpression to reject updates if the order status changed during the race.

This prevents concurrent update requests from modifying the order after payment processing begins.

Part 7) Code / Config Changes

Resource 1 changed

AWS Lambda function: DVSA-ORDER-BILLING
File: order_billing.py

Before fix — order_billing.py

Before remediation, the billing function allowed billing to continue when the order status was less than 120:

```
status = int(json.dumps(response["Item"]['orderStatus'], cls=DecimalEncoder))
if status < 120:
```

This was weak because billing did not immediately lock the order. A concurrent update request could modify the item list while payment was being processed.

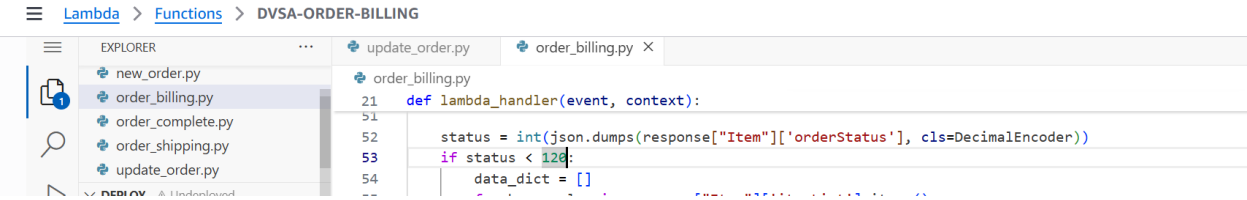

Figure 6. Before fix: billing allowed orders with orderStatus < 120.

After fix — order_billing.py

The condition was changed so billing starts only when the order is open:

```
status = int(json.dumps(response["Item"]['orderStatus'], cls=DecimalEncoder))
if status == 100:
```

Then an immediate lock was added:

```
try:
   table.update_item(
      Key={
         "orderId": orderId,
         "userId": userId
      },
      UpdateExpression="SET orderStatus = :processing",
      ConditionExpression="orderStatus = :open",
      ExpressionAttributeValues={
         ":processing": 200,
         ":open": 100
      }
   )
except Exception as e:
   print("Order lock failed:", str(e))
   res = {"status": "err", "msg": "order already locked"}
   return res
```

This changes the order status to 200 as soon as billing starts.

Figure 7. After fix: billing immediately locks the order by setting orderStatus = 200.

The final payment update was also protected with a condition expression:

```
expression_attributes = {
':orderstatus': res['status'],
':paymentTS': ts,
':total': Decimal(cartTotal).quantize(TWOPLACES),
':token': res['confirmation_token'],
':processing': 200
}

response = table.update_item(
Key=key,
UpdateExpression=update_expression,
ConditionExpression="orderStatus = :processing",
ExpressionAttributeValues=expression_attributes
)
```

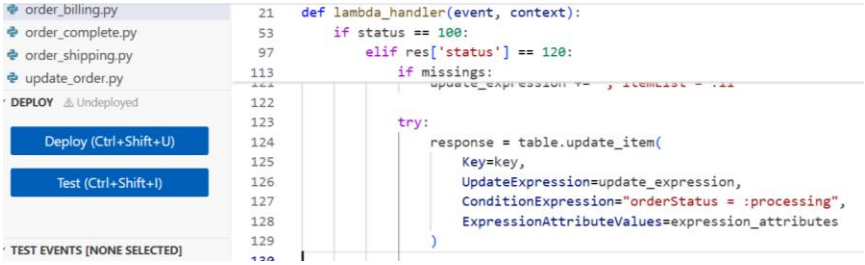

Figure 8. After fix: final payment update only succeeds if the order is still in processing state.

Resource 2 changed

AWS Lambda function: DVSA-ORDER-UPDATE
File: update_order.py

Before fix — update_order.py

Before remediation, the update function only blocked updates when the order status was greater than 110:

```
if response["Item"]["orderStatus"] > 110:
res = { "status": "err", "msg": "order already paid" }
return res
```

This left a race window where an update request could still modify the order while billing was starting.

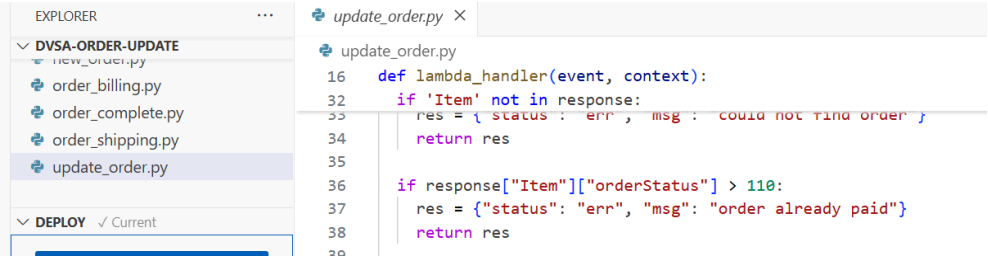

Figure 9. Before fix: update only rejected orders after orderStatus > 110.

After fix — update_order.py

The update function was changed so updates are only allowed when the order is still open:

```
if response["Item"]["orderStatus"] != 100:
res = { "status": "err", "msg": "Order cannot be modified after billing has started" }
return res
```

The DynamoDB update was also protected with a condition expression:

```
response = table.update_item(
  Key={"orderId": orderId, "userId": userId},
  UpdateExpression=update_expr,
  ConditionExpression="orderStatus = :open",
  ExpressionAttributeValues={
    ':itemList': itemList,
    ':open': 100
  }
)
```

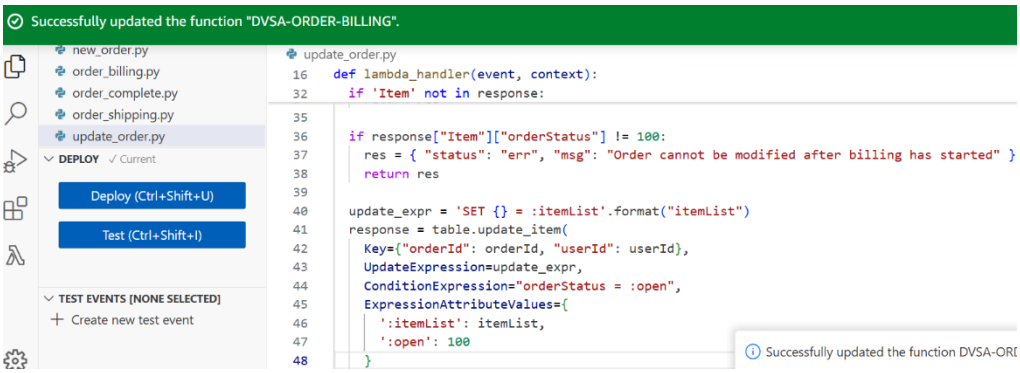

Figure 10. After fix: update requests are only allowed while the order is open.

What was changed

- Billing now starts only if orderStatus == 100.
- Billing immediately locks the order by setting orderStatus = 200.
- The billing lock uses DynamoDB ConditionExpression="orderStatus = :open".
- The final payment update uses DynamoDB ConditionExpression="orderStatus = :processing".
- Update requests are rejected unless the order is still open.
- The update function uses DynamoDB ConditionExpression="orderStatus = :open".

Why this fixes the issue

Before the fix, billing and update requests could overlap. Billing calculated the total using the original item quantity, while a concurrent update request changed the item quantity before the order finished processing.

After the fix, billing locks the order immediately. Once the order is locked, update requests are rejected. DynamoDB condition expressions enforce the rule at the database level, so even if two Lambda executions happen at the same time, the database will only allow updates when the order is in the correct state.

Part 8) Verification After Fix

After applying the fix, I created a fresh order and repeated the same race-condition test.
The post-fix test order was:

f5755b7b-3e02-44ce-b54a-c3b406be99c8

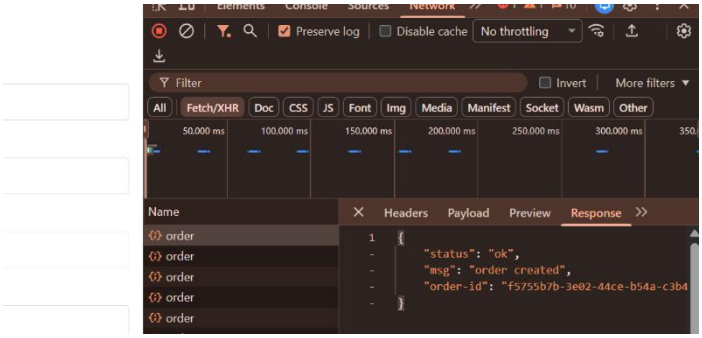

Figure 11. Fresh post-fix order created for the race-condition retest.
The same PowerShell race test was used. The billing request and update request used the same order ID. The update request again attempted to change item 1012 to quantity 10.

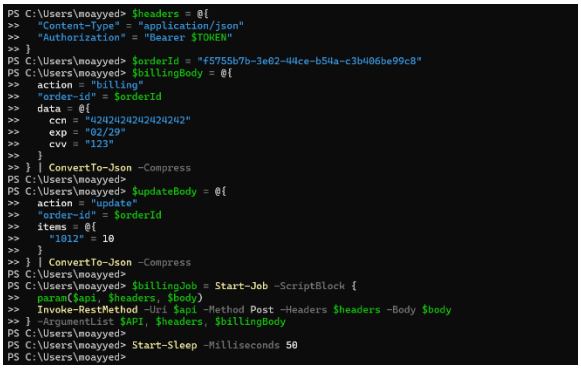

Figure 12. Post-fix race-condition retest setup.

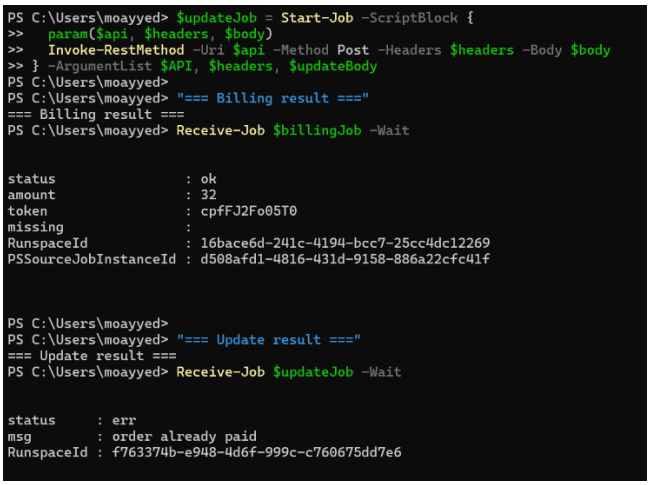

Figure 13. Post-fix retest result showing the concurrent update request was rejected.
This confirms that the race condition no longer succeeds. Before the fix, the update request returned cart updated and DynamoDB stored quantity 10 with total amount 33. After the fix, the update request was rejected, preventing the inconsistent state.
Normal application behavior still works because users can still update their cart before billing starts, and billing still works for valid open orders.

What changed:

Before the fix, the application allowed the billing and update operations to overlap in an unsafe order. The billing request could charge the original amount, while a nearly simultaneous update request could still modify the stored item quantity. This created an inconsistent final state, such as itemList = 10 while totalAmount = 33.

After the fix, the backend enforces the order workflow more strictly. Billing is only allowed when the order is still open, and once billing starts, the order is moved into a non-editable processing state. The update function now rejects changes after the order is no longer open. This prevents the item list from being changed after payment has started.

Legitimate behavior:

Normal user behavior still works after the fix. A user can still add an item to the cart, create a new order, and proceed to billing. The post-fix test showed that the billing request still succeeded and returned:

status: ok
amount: 32

This confirms that valid billing was not broken. The only behavior blocked was the unsafe update attempt after billing had started. The update request returned:

status: err
msg: order already paid

So legitimate checkout still works, but the logic flaw that allowed modifying the order after payment began is now blocked.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Logic Vulnerability	Orders should only be updated while they are open. Once billing starts, the order must be locked and the final total must match the final item list.	DVSA cart, DevTools order creation response, PowerShell billing/update requests, Lambda code, DynamoDB order record.	User creates an order with quantity 1, pays the correct total, and the final order matches the amount charged.	Billing and update requests are sent close together. Billing charges for one item, while update changes the item quantity to 10. DynamoDB stores itemList = 10 and totalAmount = 33.

Table B — Deviation analysis, fix, and verification

Vulnerability	Deviation	Class	Fix Applied	Post-Fix Verification	Optional Latency Before / After Logging
Logic Vulnerability	The system allowed order updates during billing, violating the rule that the item list and charged total must stay consistent once payment starts.	Intentional misuse / security-relevant abuse	Added order locking in order_billing.py, changed billing to start only when orderStatus == 100, added DynamoDB condition expressions, and changed	Post-fix retest showed billing succeeded, but the concurrent update request was rejected. The mismatch itemList = 10 with totalAmount = 33 no longer occurred.	Not measured.

			update_order.py to allow updates only when orderStatus == 100.		

9.1 Intended Logic and Security Rule(s)

The intended logic of the DVSA order workflow is that users can modify an order only while it is still open. Once billing starts, the order should be locked so that the item list and payment amount remain consistent.

Security rules:

- Orders can only be modified while orderStatus == 100.
- Billing should only start when the order is open.
- Once billing starts, the order should move to a non-editable state.
- The total charged must match the final item list.
- Concurrent requests must not create inconsistent order data.

9.2 Evidence Sources and Behavior Trace

Evidence sources used:

- DVSA cart page
- Browser DevTools Network tab
- PowerShell request output
- AWS Lambda code
- AWS DynamoDB order record

Normal behavior:

- A user adds one item to the cart.
- The cart shows quantity 1 and total price around $33.
- The order is created successfully.
- Billing should charge for the same quantity stored in the final order.

Exploit behavior before fix:

- The billing request and update request were sent close together.
- The billing request succeeded and charged 33.
- The update request also succeeded and changed item 1012 to quantity 10.
- DynamoDB showed itemList → 1012 = 10 while totalAmount = 33.

Post-fix behavior:

- The billing request succeeded.
- The concurrent update request was rejected.
- The inconsistent state was prevented.

9.3 Deviation Analysis and Classification

The exploit violates the intended rule that an order must not be modified after billing starts. The system allowed a concurrent update request to change the item quantity while billing was being processed. This caused the final database state to show more items than the user paid for.

This is security-relevant because it allows a user to intentionally manipulate timing to gain more items without paying the correct amount.

Classification:
Intentional misuse / security-relevant abuse

The vulnerability is also a business logic flaw because the application did not enforce the correct order-processing sequence.

9.4 Explainable Fix and Post-Fix Validation

The incorrect assumption was that billing and update requests would be processed in a safe sequential order. In a serverless environment, Lambda functions can run concurrently, so the backend must enforce order state transitions safely.

The fix was applied in both billing and update logic.

In order_billing.py, billing now starts only when the order is open. It immediately changes the order status to processing using a conditional DynamoDB update. This locks the order before payment processing continues.

In update_order.py, updates are allowed only when the order is open. A DynamoDB condition expression ensures that updates fail if the order status changes during concurrent execution.

Post-fix validation confirmed that the same race test no longer succeeds. Billing still works, but the concurrent update request is rejected. This prevents the mismatch where the order quantity becomes 10 while the amount charged remains 33.

Part 10) Takeaway / Lessons Learned

This lesson shows that business logic vulnerabilities can happen even when authentication and basic API access controls are working. The issue was not that the user was unauthenticated; the issue was that the backend allowed valid requests to be processed in an unsafe order.
In serverless applications, race conditions are especially important because Lambda functions can run independently and concurrently. If the application does not enforce state transitions at the database level, concurrent requests can create inconsistent and insecure outcomes.
The key lesson is that workflow rules must be enforced on the server side using state validation, early locking, and database-level conditional updates. A user-facing order update function should never be able to modify an order once billing has started.